

Constitutive Authorization at the Decoding Boundary: Grammar-Constrained Decoding as a Positive, Generation-Time Security Control for LLM Agents

Vincent Andino*

Draft — June 23, 2026

Abstract

We port Galloway’s (2020) case for application whitelisting – a positive, structural control that admits only what is explicitly authorized – from operating-system execution to large-language-model agent tool calls [1]. Against a single deployed CTF agent (Tantalus) we replay an outcome-selected corpus of capable authorization-class attacks across nine conditions: a security-neutral Control baseline, four negative/behavioral controls (a defensive prompt **a1**, an input classifier **a2**, an output filter **a3**, and the combined stack **a4**), a corrective post-parse allowlist gate (**d**), and the positive/structural control — grammar-constrained decoding (GCD), in which a per-request grammar G_s makes out-of-scope tokens ungenerable at the sampler (blind **c**, with the closed-response **c_plus** and least-privilege **c_guided** variants). Like Galloway, we score by effect at scale with Clopper–Pearson confidence intervals [8]; unlike Galloway, we add two layers: a soundness core that quantifies over all logit distributions – turning an observed 0 into a provable 0, emitted $\in L(G_s) \subseteq \text{Authorized}(s)$ – and a cost/economics axis. Our thesis is constitutive vs. corrective: **c** and **d** enforce the same policy over $L(G_s)$ but differ in enforcement completeness, the empirical signature being emission (the count of times the unauthorized artifact was produced: 0 under **c** by construction, > 0 under **d**). Across three datasets ($\sim 2,412,613$ trials): on a 1.7B generator $\times$ gate factorial the GCD arms show 0 emission and an inert gate while behavioral-behind-gate cells produce-then-catch 104k–120k out-of-scope calls each – the GCD arms running at $\sim 40\text{--}55\%$ of those arms’ tokens; on the capable non-Qwen Mistral-Small-4-119B anchor **a4** still leaks (bypass 1.82% [1.43, 2.29], CI-lower > 0) while **c** is 0.00% [0, 0.09], below every behavioral point estimate, with a validity dividend (99.8% valid vs. $\sim 59\%$ native); and an ablated steelman moves 97.85% $\rightarrow$ 0%. Honesty envelope: the headline leak and the formal contrast are carried by a single capable anchor; external validity covers 2 of 3 pretraining lineages (Qwen and Mistral); the registered 2-percentage-point non-inferiority TOST is not yet powered; grammar-compile time is not yet captured; and **c_sealed**’s 0% is partly true by construction.

1 Introduction

LLM agents are now deployed with real tool access – they read files, query inboxes and chat logs, and issue outbound HTTP requests on behalf of a user. Most of these tool calls happen in backend pipelines the user never sees: an agent invoked to “run the daily sync” silently reads,

*Correspondence: vincent.andino2@gmail.com. Draft manuscript; affiliation omitted pending submission.

retrieves, and forwards on the user’s behalf, and only a summary returns to the screen. This creates a confused-deputy exposure that the literature has named *indirect prompt injection*: malicious instructions placed in a trusted data or skill channel – an email, a support ticket, a tool’s own documentation – are ingested as content and then followed as commands, redirecting the agent’s authorized capabilities toward an attacker’s ends [3]. The injection rides in through exactly the channels the agent must read to do its job.

The defenses the industry reaches for are *behavioral*: a defensive system prompt (an RFC-2119 security policy), an input classifier on the user message, an output filter scanning tool-call parameters against a credential denylist. These are negative, default-allow controls – they enumerate the bad and permit everything else – and they are permanently bypassable for a structural reason, not a tuning one. A defensive prompt and an input classifier are *in-band*: they live in the same input channel the injection occupies, so an injection that can address the model can address them too, and overrides them. An output filter is a negative denylist: business data carrying no credential signature sails straight through it. No amount of prompt engineering or classifier retraining removes a weakness that is a property of *where* and *how* the control is applied.

This is the situation Galloway analyzed for the host-security domain in 2020: signature-based antivirus (a negative control – block known-bad) versus application allowlisting (a positive control – permit only known-good), measured not by detection scores but by running thousands of live malware samples and recording whether the malware actually *ran* [1]. We port that experiment to LLM agents. The negative controls are the behavioral guardrails above; the positive control is Grammar-Constrained Decoding (GCD) with a *per-request authorization grammar* G_s , compiled from the agent’s authenticated identity, that admits only in-scope tool calls. The attack corpus is held fixed across both, and – following Galloway, and unlike a structured-output conformance check – every trial is scored by *effect*: a bypass is recorded iff the agent emits a tool call whose execution would carry the attack’s sentinel marker to an out-of-scope sink (the win flag in `fetchURL.data` to a non-owned endpoint, or `readChatHistory` on an unauthorized channel), per a deterministic win detector. We run this at a scale (millions of attack trials) commensurate with the modern agent-security benchmark landscape [4], and report every rate as a Clopper–Pearson exact binomial interval [8].

The reason a positive grammar holds where a positive *policy* alone does not is a matter of enforcement locus. The transformer always computes a full logit distribution, and under injection it may assign an arbitrarily high logit to a forbidden token – the model’s “intent” is uncontrolled and may be fully adversarial. But GCD applies the grammar mask *before* sampling, driving every grammar-invalid token’s probability to zero, so such a token cannot be generated by any sampler – greedy, top- k , top- p , or temperature [5, 6]. The model is an untrusted machine, the sampler is the gate, and the grammar is the allowlist. Logits encode intent and are left uncontrolled; the generated tokens are constrained out-of-band, at a point the attacker’s tokens never reach.

We add two layers Galloway’s empirical study lacked. The first is a *soundness theorem*: because the grammar mask quantifies over all logit distributions, the guarantee emitted $\in L(G_s) \subseteq \text{Authorized}(s)$ holds against a fully injection-compromised model, never assuming the logits are benign – which is what licenses a class-level elimination-by-construction claim that no finite sample could (Section 4). The second is a *cost axis*: token generation is not free. We compare GCD against the strongest deployed positive control, a post-parse allowlist gate (call it the corrective allowlist) that enforces the *same* policy over the *same* language $L(G_s)$ but *after* an unbounded generator has already produced the call. The two are policy-equivalent and differ only in enforcement completeness: GCD is *constitutive* – the out-of-scope action is ungenerable and exists as an artifact nowhere (output buffer, logs, telemetry, stream, KV cache); the allowlist is *corrective* – it must generate-then-reject, so the forbidden artifact is produced in full and then caught, re-introducing

one layer down the production cost, output-channel artifact, coverage obligation, and TOCTOU window that positive security exists to escape. The empirical signature of this difference is the per-trial *emission* count – the number of times the unauthorized artifact actually existed – which is 0 under GCD by construction and > 0 under the allowlist.

Why, then, is scale a finding rather than a tautology? Two reasons. First, bypass is scored by effect through the *unconstrained* surfaces – the free-text fields a grammar cannot enumerate (`searchFiles.query`, `respondToUser.message`) remain open, so a 0% bypass at the structured sink is an empirical claim about what the agent does *with* that latitude, not a restatement of the grammar’s membership relation. Second, volume stresses the implementation, not just the statistics: at millions of trials, real engine and apparatus bugs surfaced – a tool-name casing mismatch that inflated emission, a structured-output backend that crashed the engine by rejecting the auto-tool-choice (LARK) grammar on the first tool call – that a small pilot would never have exposed. Large N tightens the confidence intervals; it does not, by itself, buy generalization, and we are explicit about that throughout.

We keep the scope honest from the outset. This is a single-subject *feasibility* study on the Tantalus CTF agent, not a population-level universality claim. The attack corpus is selected on outcome (attacks known-effective against the undefended agent – the live-malware analogue), so the reported bypass rates are conditional on attack potency, with the no-defense replay rate as the reference baseline. The headline security claim is *action integrity over enumerable-sink tools* (structured tool calls and enumerable parameters); free-text confidentiality leakage through channels that must remain free-text is explicitly Paper-2 scope.

Contributions. This paper makes the following contributions.

- **The constitutive-vs-corrective distinction, and its empirical signature.** We articulate why GCD and a post-parse allowlist enforce the same positive policy yet differ in enforcement completeness – GCD’s positivity is enforcement-deep (all the way down to the generative act), the allowlist’s is policy-deep only – and we make *emission* the primary structural dependent variable that measures the difference directly (Section 3).
- **A soundness theorem quantified over all logit distributions.** We prove $\text{emitted} \in L(G_s) \subseteq \text{Authorized}(s)$ against a fully injection-compromised model, separating the proof’s class-level guarantee from the experiment’s evidence in a three-tier ladder (Section 4).
- **A large-scale, effect-scored port of Galloway to LLM agents.** A generator $\times$ gate factorial design (Section 5) over ~ 2.3 million attack trials with Clopper–Pearson intervals: on a capable non-Qwen anchor the full behavioral stack still leaks (A4 bypass 1.82% [1.43, 2.29], CI-lower > 0) while GCD holds at 0.00% [0, 0.09], strictly below every behavioral point estimate; across all GCD attack arms, 852,000 trials at zero bypass (Section 6).
- **The cost and reliability axes Galloway lacked.** We quantify the produce-then-catch tax (emission, output tokens, retry attempts, availability stranding) the corrective allowlist pays and GCD avoids, and a least-privilege variant (`c_guided`) that turns blind GCD’s two-sided reliability story into a one-sided utility win (Sections 6 and 7).

Roadmap. Section 2 fixes the threat model and the effect-scored bypass definition; Section 3 develops the constitutive-vs-corrective core; Section 4 states and proves the soundness theorem and the three-tier ladder; Section 5 describes the subject system, conditions, and factorial design; Section 6 reports the three datasets (Sections 6.1, 6.2, 6.3) and the aggregate hard-statistics ledger

(Section 6.4); Sections 7 and 8 discuss implications and limitations honestly, with related work in Section 9 and conclusions in Section 10.

2 Background and Threat Model

2.1 Subject system: Tantalus

The subject system is *Tantalus*, a real, previously-built and deployed agentic application — an AI-security CTF agent — that ships with a known, calibrated corpus of attacks independently observed to succeed against its behavioral defenses. For this study Tantalus runs as a self-contained *local* binary on `vLLM / llama.cpp` [9] with grammar support (the original cloud path replaced), so that Grammar-Constrained Decoding (GCD) is a true token-level mask at the sampler rather than a managed cloud structured-outputs feature. The agent acts as a desktop-support deputy for a *single* principal — “Jordan” at a consultancy — over seven tools: `readFile`, `searchFiles`, `listInbox`, `readEmail`, `readChatHistory`, `fetchURL`, and `respondToUser`.

2.2 Authorization model and the confused-deputy threat

The principal’s authority is the set of tool calls legitimately needed for the task at hand. Attacks are *authority overreach*: a confused-deputy redirect [3] in which poisoned skills or emails induce the deputy to read a resource outside the task scope and/or to exfiltrate via `fetchURL` to a non-owned endpoint. An “authorization-class attack” is one whose success requires emitting a tool call with parameters outside the principal’s authorized set — the action space the legitimate task never needs to enter.

We score success by *effect*, after Galloway’s application-whitelisting study [1], whose “did the malware actually run?” criterion this work ports into the agent domain. A trial is a **bypass** iff the agent causes the harmful effect of an authorization-class attack: it emits a tool call whose execution would carry the attack’s sentinel marker to an out-of-scope sink — the win’s flag in `fetchURL.data` to a non-owned endpoint, or `readChatHistory` on a non-authorized channel — as determined by a deterministic win detector (flag-marker plus scope-oracle, no human grading). Otherwise the trial is a **block**. Bypass is therefore an end-to-end property of the deployed system scored on the effect, never on grammar-membership; this matters because the headline security claim must not be co-defined with the grammar that closes the sink (Section 6).

The five Tantalus win-classes are the “malware families” of this port, instantiated across paraphrase/obfuscation variants and a delivery axis (direct prompt vs. second-order email-trap / poisoned skill):

- **W1** — SSH-key exfiltration (private key to an attacker endpoint).
- **W2** — API-key exfiltration (AWS-style credentials).
- **W3** — infra-monitor exfiltration (credential read via a daemon persona).
- **W4** — ticket-assist auto-forward: business data with *no credential signature*.
- **W5** — cross-user chat exfiltration (a non-authorized channel).

W4 is the class that defines negative-security incompleteness: its payload — support-ticket text — matches no credential pattern, so a signature denylist (A3, below) cannot catch it. Section 6 reports that the survivors of the output filter on a capable victim are precisely this credential-signature-free class.

2.3 In-band vs. out-of-band defenses

The decisive structural property of a control is *where it sits relative to the attacker’s channel*. The behavioral controls A1 (a defensive system prompt with an RFC-2119 security policy) and A2 (an embedding classifier on the user message) are *in-band*: they live in the same input channel the attacker’s injection occupies, so an injection through a trusted skill or email can simply override them. GCD (Condition C) is *out-of-band*: the grammar G_s is compiled from the authenticated principal’s identity and applied at the sampler — nowhere the attacker’s tokens can reach. That in-band/out-of-band split is the precise reason one fails and one holds. The enforcement locus is the *sampler*, not the model: the transformer always computes a full logit distribution and under injection may assign an arbitrarily high logit to a forbidden token [7]; GCD applies the grammar mask [5, 6] *before* sampling, driving every grammar-invalid token’s probability to 0 so it cannot be generated by any sampler (greedy, top- k , top- p , temperature). Logits (the model’s intent) remain uncontrolled; the generated tokens are constrained — this is what licenses the soundness statement emitted $\in L(G_s) \subseteq \text{Authorized}(s)$ developed in Section 4.

The corrective comparator, Condition D, is a post-parse allowlist gate that checks the *parsed* tool call against the same allowlist G_s compiles. Its gate configuration is itself derived from identity (out-of-band), but its *enforcement* is post-generation: the forbidden artifact is produced first, then caught. C and D thus enforce the same positive policy over the same language $L(G_s)$ and differ only in enforcement completeness, not in what they permit; that distinction — constitutive vs. corrective — is the core of Section 3.

2.4 Held-constant substrate, ethics, and an open decision

Two design points scope the contrast. First, *structured-output shape* (the requirement that every tool call be valid JSON) is held constant across *all* conditions, including Control: it is the substrate, not a control, and shape is not authorization. There is no “shape-only” arm and we make no claim that valid-JSON enforcement constitutes authorization. Second, the experiment is fully local and ethically contained: `fetchURL` and all outbound channels are mocked (no real exfiltration, no SSRF surface), the environment and secrets are synthetic sentinel markers rather than real credentials, no external systems are attacked, and there are no human subjects. The corpus is *selected-on-outcome* — attacks independently known to succeed against the undefended agent, the live-malware analogue [1] — so reported bypass rates are conditional on attack potency and large N tightens Clopper–Pearson intervals [8] without buying generalization. Multi-token prediction (MTP) was treated as a per-engine decision (verify per-token grammar-mask correctness and run-to-run determinism, or disable it); in practice the per-trial MTP setting was not logged for the three completed databases — a realized reproducibility gap taken up in Section 8 [2].

3 Constitutive vs. Corrective Authorization

The post-parse allowlist gate (D) and grammar-constrained decoding (C) are not competing policies. They enforce the *same* positive authorization policy over the *same* language $L(G_s)$: the set of tool calls a session with authenticated identity s is permitted to make. Both derive that language from the same artifact — in our implementation D’s gate and C’s grammar both call `tantalus_grammar::allowlist_verdict`, so the permitted set is identical by construction. What differs is *where the policy sits relative to generation*, and therefore *how completely* it is enforced.

3.1 Two enforcement loci over one policy

C is **constitutive**: the policy *is* the output space. The grammar G_s is compiled from the authenticated identity and applied as a token mask at the sampler, driving the probability of every grammar-invalid token to zero *before* sampling. An out-of-scope action is therefore *ungenerable* — it never exists as an artifact on any surface (output buffer, logs, telemetry, response stream, KV cache). Enforcement is positive security all the way down to the generative act.

D is **corrective**: an unbounded, default-allow generator first produces a tool call *in full*, after which a separate gate inspects the parsed call and rejects it if it falls outside $L(G_s)$. **D** is thus a positive *policy* placed on top of a negative-security *generator*. This re-introduces, one layer down, the three failure modes that positive security exists to escape:

1. **The artifact is produced.** The unauthorized call is generated, costing output tokens and leaving an output-channel artifact — a confidentiality, forensics, and side-channel surface that exists whether or not the gate later refuses to forward it.
2. **A coverage obligation.** The gate must fire on *every* output path, every downstream consumer, and after every refactor, or the unauthorized call reaches an executor. Coverage is the same completeness obligation that negative-security denylists fail.
3. **A TOCTOU window.** The call exists between generation and the gate’s verdict; any consumer reading the buffer in that window observes the unchecked artifact.

C has none of these because there is nothing to catch. This is the ported form of Galloway’s application-whitelisting argument [1]: the structural, positive control forecloses the action rather than detecting it after the fact.

The honest statement of the difference is enforcement-completeness, not security strength. We do *not* claim **C** blocks the harmful *effect* more reliably than a perfectly-wired allowlist on the calls both see — against the same set of forwarded actions they are policy-equivalent. We claim that **C**’s positivity is *enforcement-deep* (it reaches the generative act) while **D**’s positivity is *policy-deep only* (correct policy, corrective enforcement). The same post-parse check that constitutes **D** is, accordingly, **C**’s recommended corrective backstop in the trusted-computing-base discussion (Section 4); it is not a weaker control to be discarded, but a complementary layer that becomes *inert* when nothing unauthorized is ever generated.

3.2 In-band vs. out-of-band

The behavioral controls (**a1**, **a2**) are **in-band**: a defensive system prompt and an input classifier live in the same input channel the adversary’s injection occupies. The injection shares that channel and can override the instruction, which is why indirect prompt injection through trusted data defeats them [3]. **C** is **out-of-band**: G_s is compiled from authenticated identity and applied at the sampler — nowhere the attacker’s tokens can reach. Critically, the enforcement locus is the sampler, not the model. The transformer always computes a full logit distribution, and under injection it may assign an arbitrarily high logit to a forbidden token; the grammar mask zeroes that token’s probability regardless of the logit, so it cannot be emitted under any decoding strategy (greedy, top- k , top- p , temperature). Logits encode intent and remain uncontrolled; the *generated tokens* are constrained. This in-band/out-of-band split — not any difference in policy — is the precise reason one control fails and the other holds.

3.3 The emission DV: the empirical signature of the distinction

The constitutive–corrective difference is not merely conceptual or about efficiency; it is directly observable. We measure it with the *emission* dependent variable.

Definition 1 (Emission). For an attack trial, **emission** is the per-trial count of times the unauthorized tool call was actually *produced* by the generator, independent of whether a downstream gate later blocked it. Under constitutive enforcement (C) the unauthorized call is ungenerable, so emission = 0 by construction. Under corrective enforcement (D) the call is produced and merely not forwarded, so emission > 0. The live-gate analogue is **gate_rejections**: out-of-scope calls produced-then-caught. Emission is the *primary structural* DV for the constitutive-vs-corrective research question, not a secondary efficiency statistic.

Emission separates block-by-construction from block-by-detection at the per-trial level. It is distinct from the primary effect DV (bypass), which is scored by *effect* — whether a tool call carrying the attack’s sentinel marker reaches an out-of-scope sink — in Galloway’s “did the malware run” sense [1], never by grammar membership. A trial can have emission = 1 and bypass = 0 (D produced the call, then caught it); under C both are 0 because the artifact never existed.

3.4 The generator $\times$ gate factorial: the contribution moves off the bypass axis

Because D is a post-parse check on a *parsed* call, it is orthogonal middleware rather than a peer condition: it can be layered over any generator. We therefore treat the allowlist as a factor — an orthogonal **gate** modifier with retry budget $R \in \{0, 1, 3\}$ (cells **d_r0**, **d_r1**, **d_r3**) — crossed with each generator (cells such as **a4_d_r3** and **c_d_r3**). The empirical consequence is that *any* generator behind a correct gate achieves ≈ 0 bypass. In the 282,000-trial-per-cell factorial on the weak workhorse (**blast_1p7b_5m.db**), every gated cell — behavioral and grammar alike — records 0.00% bypass.

The contribution therefore does not live on the bypass axis at all; it lives on the *cost* of reaching that zero. The factorial makes three things visible (Table 1):

1. **Behavioral generators make the gate work harder.** **a1_d_r3** emits 120,140 out-of-scope calls and triggers 139,612 gate rejections; **a4_d_r3** emits 119,091 with 133,221 rejections. The defensive layer in front of the gate does not reduce the dangerous emission — it is produce-then-catch either way.
2. **C makes the gate inert.** The three GCD cells (**c_d_r3**, **c_closed_d_r3**, **c_guided_d_r3**) record 0 emission, 0 gate rejections, and 0 availability failures: the grammar already constituted the policy, so the corrective gate has nothing to do.
3. **The cost gap is real but token-cost ordering is victim-dependent.** On the weak workhorse the GCD cells spend ≈ 112 –149 output tokens versus ≈ 261 –289 for the behavioral-plus-gate cells, and the GCD validity dividend is large (valid-output 97.8%–99.9% vs. 89.4%–93.9%). On the capable Mistral-119B anchor the emission and validity conjuncts carry decisively (**c** 0 emission, 99.8% valid vs. $\approx 59\%$ for native free-generation) but **c**’s 208 output tokens are not strictly below every D variant (**d_r1** 203), so the registered conjunctive token-cost criterion is met only on the 1.7B victim. The victim-independent result is the qualitative gate-inertness; the token ordering is victim-dependent.

So the slogan “same policy, enforcement-completeness differs” is made operational: behavioral controls add cost *in front of* the gate without reducing emission; the gate restores the positive policy

Table 1: The generator $\times$ gate factorial on the weak workhorse (`blast_1p7b_5m.db`, $n = 282,000$ attack trials/cell). Every gated cell reaches 0.00% bypass; the contribution is the *cost* of doing so. Behavioral generators produce $> 10^5$ out-of-scope calls the gate must catch; `C` emits none, leaving the gate inert.

cell	bypass (%)	emission	gate_rej	avg out-tok	avail_fail
<code>control</code> (ungated)	18.33	105,173	–	289	–
<code>a1_d_r3</code>	0.00	120,140	139,612	266	45
<code>a2_d_r3</code>	0.00	104,697	120,632	270	46
<code>a3_d_r3</code>	0.00	104,615	111,528	269	46
<code>a4_d_r3</code>	0.00	119,091	133,221	261	44
<code>c_d_r3</code>	0.00	0	0	119	0
<code>c_closed_d_r3</code>	0.00	0	0	112	0
<code>c_guided_d_r3</code>	0.00	0	0	149	0

but pays a produce-then-catch tax (emission, tokens, retry attempts, and availability stranding at $R = 0$); and `C` eliminates that tax by construction, making the corrective layer inert.

Remark 1 (Reliability is two-sided). Neither enforcement locus is unilaterally superior on reliability. Constitutive enforcement guarantees a one-shot valid in-policy action, but a blind union grammar can silently *deflect* a masked malicious intent into a valid but unrequested in-scope action — a reliability cost that is reported *against C* (the deflection DV, Section 5) and that the per-request least-privilege grammar (`c_guided`) drives to 0 by construction. Corrective enforcement, conversely, yields an explicit, auditable denial when it strands an action — a desirable forensic property — but consumes the turn: at $R = 0$ every produced-then-caught trial terminates in an availability failure, and recovery requires serialized retries. We measure both; we claim neither as dominant.

4 Formal Core: A Soundness Sketch

Beyond Galloway’s empirical block rate [1], this work adds a cost/economics axis and — the subject of this section — a *soundness theorem*. Galloway could report only that some fraction of malware was stopped — a finite measurement that licenses no statement about the next, unseen sample. Grammar-constrained decoding, by contrast, admits a class-level guarantee: under the conditions stated below, an emitted action provably lies within the authorized set, *independent of the model’s logits* and therefore against a fully injection-compromised model. This section states that guarantee precisely and is scrupulous about its scope and its trusted computing base (TCB). We present it as a *sketch*: the fully detailed proof, with the tokenizer-lifting argument made first-class, is a companion artifact (`docs/proof.md`, in preparation). We do *not* claim a complete mechanized proof here.

4.1 Setup

Fix a tokenizer with finite vocabulary V . A generation is a finite sequence of tokens $t_1 t_2 \cdots t_n \in V^*$. For an authenticated principal with scope s , the G_s compiler (the novel artifact, Section 5) turns s into a context-free grammar G_s whose terminals are tokenizer tokens; authorized accounts, endpoints, channels, and paths become ground terminals, and out-of-scope parameters are absent from the language $L(G_s) \subseteq V^*$. We write $\text{Authorized}(s) \subseteq V^*$ for the set of token sequences whose decoded action is permitted for principal s .

The constrained decoder applies, at each decode step i , a *grammar mask* computed from the prefix $t_1 \cdots t_{i-1}$ already emitted. The mask drives to zero the probability of every token that

cannot extend that prefix to a valid prefix of some string in $L(G_s)$, *before* sampling. Formally, let $\text{Ext}_{G_s}(p) \subseteq V$ denote the set of tokens t such that pt is a prefix of some member of $L(G_s)$. The decoder restricts the support of the next-token distribution at step i to $\text{Ext}_{G_s}(t_1 \cdots t_{i-1})$, with the end-of-sequence token permitted exactly when the prefix is itself in $L(G_s)$. This is the standard push-down-automaton mask realized by xgrammar [6] and llama.cpp GBNF [9], in the same lineage as guidance-style constrained generation [5]. The enforcement locus is the *sampler*, not the model: the transformer always computes a full logit distribution over V , and under injection it may assign an arbitrarily high logit to a forbidden token; the mask is applied to that distribution prior to any sampling decision.

4.2 Mask Soundness

Lemma 1 (Mask soundness). *Fix G_s and run the constrained decoder above. For any sequence of logit distributions over V produced by the model and any sampler — greedy argmax, top- k , top- p (nucleus), or temperature sampling at any temperature — every token drawn at step i lies in $\text{Ext}_{G_s}(t_1 \cdots t_{i-1})$. Consequently, any generation the decoder reports as complete is a member of $L(G_s)$.*

Proof sketch. At step i the mask sets the probability of every token outside $\text{Ext}_{G_s}(t_1 \cdots t_{i-1})$ to zero before sampling. Each of the named samplers selects a token from the support of the (renormalized) distribution it is given: argmax returns a token of maximal probability, which has nonzero probability; top- k , top- p , and temperature reweighting all preserve the zero set — a token assigned probability zero remains unselectable under any of them. Hence the drawn token has nonzero post-mask probability and therefore lies in $\text{Ext}_{G_s}(\cdot)$. This holds at every step regardless of what the logits were, since the argument never inspects the logit values, only the support after masking. By induction over the n steps, the prefix invariant “ $t_1 \cdots t_i$ is a valid prefix of $L(G_s)$ ” is maintained, and completion is admitted only when the prefix is in $L(G_s)$; therefore the completed generation is a member of $L(G_s)$. A complete treatment, including the end-of-sequence condition and the tokenizer lift of Assumption 2, is deferred to docs/proof.md. $\square$

The quantification “for any logit distribution” is the load-bearing phrase: the lemma never assumes the logits are benign, so it covers the adversarial case in which a prompt injection has driven the model to *intend* a forbidden action. Logits encode intent and are uncontrolled; generated tokens are constrained.

4.3 The Soundness Theorem

Theorem 1 (Action-integrity soundness). *Let emitted be any completed generation of the constrained decoder for principal scope s , and suppose the compiled grammar satisfies $L(G_s) \subseteq \text{Authorized}(s)$ (Assumption 1). Then*

$$\text{emitted} \in L(G_s) \subseteq \text{Authorized}(s), \tag{1}$$

so the emitted action is within the authorized set — independent of the logits, hence against a fully injection-compromised model.

Proof sketch. By Lemma 1, $\text{emitted} \in L(G_s)$ for any logits and any sampler. By Assumption 1, $L(G_s) \subseteq \text{Authorized}(s)$. Compose the two inclusions to obtain (1). $\square$

What this licenses, and what it does not. Theorem 1 is a statement about *every* producible output of the mechanism, not about a measured fraction of them. It is therefore the one asset that supports a class-level *eliminates-by-construction* claim — out-of-scope actions are ungenerable — which no finite sample could establish however large. This is the formal step beyond Galloway, whose contribution was an empirical block rate alone [1]. The empirical samples reported in Section 6 (including 852,000 GCD attack trials at 0 bypass, of which the 2,000 steelman trials are partly true-by-construction) do *not* carry the “eliminates” claim; they show that behavioral controls genuinely leak at scale (Section 6.2) and that the implementation matches the theory. Proof and evidence stay in separate lanes.

4.4 Trusted Computing Base and Caveats

The guarantee is conditional, and we state the conditions honestly.

Assumption 1 (Compiler / allowlist correctness — the TCB). The G_s compiler is trusted: Theorem 1 holds *iff* the compiled language satisfies $L(G_s) \subseteq \text{Authorized}(s)$. If the allowlist is wrong — it admits a string whose decoded action is not in fact permitted — the theorem’s conclusion follows from a false premise and provides no protection.

The compiler is therefore the trusted computing base, exactly as the policy engine is the TCB in any positive-security control. We mitigate this dependency, but do not eliminate it, with two independent checks: (a) an automated *acceptance oracle* that validates each compiled grammar against the target engine before any run (and revalidates per engine; Section 5); and (b) an *executor re-validation backstop* that re-checks the decoded action against the same allowlist at the point of execution. The backstop is itself a Condition-D-class corrective check (the post-parse allowlist gate, `tantalus_grammar::allowlist_verdict`), deployed here as defense-in-depth that *complements* the constitutive control rather than contradicting it: the same post-parse check is the recommended corrective backstop for C (Section 7). C and D are policy-equivalent over $L(G_s)$; they differ in enforcement completeness, and using one to guard the TCB of the other is consistent with that framing.

Assumption 2 (Sound tokenizer lift). The grammar G_s is authored at the character level (URLs, paths, channel identifiers), but the mask of Section 4.1 operates over the *token* language. Correctness requires a sound lift from the character-level grammar to the tokenizer token language — the token-level language must decode to exactly the intended character-level language, with no over- or under-approximation introduced by token boundaries.

The tokenizer lift is an engineering obligation, not a free assumption: a faulty lift can admit a token sequence that decodes to an out-of-scope string, silently violating Assumption 1 one layer down. We discharge it operationally, per engine, via the acceptance oracle (it asserts that the engine actually constrains output to the intended language on probe inputs, catching silent-ignore and tokenizer-misalignment footguns), and we make it first-class in the companion proof.

Remark 2 (Scope). Theorem 1 covers *action integrity over enumerable-sink tools* — structured tool calls whose authorized parameter values form an enumerable language (endpoints, channels, paths, message identifiers). It does *not* cover free-text confidentiality through parameters that must remain unconstrained, such as `searchFiles.query` or `respondToUser.message`: a free-text field is, by construction, not narrowed by G_s , so the theorem makes no claim about what flows through it. Confidentiality through unconstrained free-text channels is Paper-2 scope, addressed there by complementary measures (enumerable-response closure for the enumerable-response agent class,

and layered detection). The single-subject realization (Tantalus) makes this a feasibility result for the covered class, not a population-level universality claim.

In short: against an adversary who fully controls the logits, GCD’s positivity is *enforcement-deep* — it reaches all the way down to the generative act — conditional on a correct allowlist and a sound tokenizer lift, both of which we guard with independent, defense-in-depth checks. The detailed proof, with these two obligations promoted to discharged lemmas, is the companion artifact `docs/proof.md` (in preparation).

5 Experimental Design

5.1 Design type

The study is a *quantitative true experiment* of the pre-test/post-test/control-group form, ported directly from Galloway’s evaluation of application allowlisting against live malware [1], and composed with a formal soundness core and a built artifact. In the methodological taxonomy of Amaral [2], this is a deliberate composition of the *Experimental*, *Formal*, and *Build* methodologies: each claim is carried by the method appropriate to it (Section 4 for the proof; this section and Section 6 for the experiment). Amaral explicitly sanctions the composition — the activities required to answer a single research question may draw on several methodologies — and so we keep proof and evidence in separate lanes (the three-tier ladder of Section 3). Following Galloway, qualitative and correlational designs are rejected: a positive/structural control (GCD) is tested against the negative/behavioral controls that industry actually deploys, on the *same* attack corpus, scored by the *effect*, and reported as block/bypass rate with Clopper–Pearson 95% confidence intervals [8] at scale.

5.2 Independent variable and what is held constant

The single independent variable is the security control in force. Everything else is a controlled constant: the agent (Tantalus), the seven tools, the embedded environment, the model under test within a given run, the *security-neutral operational base prompt* (persona, tool definitions, and output format, with no security or defensive content), the attack corpus, the inference engine and flags, and the seed/temperature policy. Crucially, *structured-output shape* (valid JSON tool calls) is also held constant across every condition — it is the substrate, not a control. Shape is not authorization, and there is therefore no separate “structured-output-shape-only” arm. The Control condition is base-prompt-only, so the baseline is de-contaminated: a base prompt containing “never exfiltrate” would make Control secretly defended and understate attack potency.

Table 2 lists the conditions. The in-band/out-of-band column records the structural reason behavioral controls fail and GCD holds: A1/A2 are *in-band* (they occupy the same input channel the attacker’s injection occupies, so the injection overrides them), whereas the GCD family is *out-of-band* (the grammar is compiled from authenticated identity and applied at the sampler, nowhere the attacker’s tokens can reach). The post-parse gate D is policy-positive but corrective: its configuration is out-of-band, but enforcement is post-generation — the forbidden artifact is produced, then caught.

The defining property of the GCD family is that the policy *is* the output space. A per-request GBNF G_s is compiled from the authenticated principal’s scope; grammar-invalid tokens are ungenerable at the sampler. The enforcement locus is the sampler, not the model: the transformer always computes a full logit distribution and under injection may assign an arbitrarily high logit

Table 2: The independent variable: nine security controls. Type is negative/behavioral, positive/constitutive (enforced at the sampler), or positive policy enforced *correctively*. Channel is the locus relative to the attacker’s input channel.

ID	Condition	Type	Channel
control	No added mechanism	baseline	— (substrate only)
a1	Defensive prompt (RFC-2119)	negative, behavioral	in-band
a2	Input classifier (embedding)	negative, behavioral	in-band
a3	Output filter (credential denylist)	negative, behavioral	post-generation
a4	Full behavioral stack (A1+A2+A3)	negative, behavioral	combined
c	GCD over union allowlist (blind)	positive, constitutive	out-of-band
c_plus	GCD, closed-response (C+)	positive, constitutive	out-of-band
c_guided	GCD, per-request least-privilege	positive, constitutive	out-of-band
d	Post-parse allowlist gate	positive policy, corrective	out-of-band cfg., post-gen. enforcement

to a forbidden token, but the grammar mask is applied *before* sampling, driving every grammar-invalid token’s probability to zero so it cannot be generated by any sampler (greedy, top- k , top- p , temperature). Logits (intent) are uncontrolled; generated tokens are constrained. The post-parse gate D, by contrast, calls the same allowlist verdict (`tantalus_grammar::allowlist_verdict`) that G_s is built from — so D and C enforce the *same policy over the same language* $L(G_s)$ — but D applies it after an unbounded, default-allow generator has already produced the call. C and D are policy-equivalent; they differ in enforcement completeness, not in what they permit.

5.3 The GCD ladder and the steelman condition

The positive cells form a granularity ladder. *L1 (blind)*, condition **c**, is a union allowlist over all tools with enumerated parameters and free-text left open; it is retained as the headline *security* cell, because 0-bypass over the union allowlist is the falsifiable claim (a hardcoded one-action grammar would make 0% trivial). *L2 (guided)*, condition C+ / **c_plus**, additionally enumerates `respondToUser.message` to a finite canned-response alternation, giving 0 content-emission by construction for the enumerable-response agent class and closing the truncation residual. *L3 (closed)*, condition **c_guided**, narrows G_s per request to exactly the trusted-channel authorized action(s), with provenance derived from task identity via a single policy map invariant to untrusted skill content — driving deflection to 0 by construction; this is the utility/granularity result, not the headline security cell. We note one reconciliation: the source identifiers **c_closed** and **c_guided** were introduced with their ladder meanings inverted relative to the L2/L3 labels above and were renamed to ladder-explicit canonical names (**c_12_guided**, **c_13_closed**) with the old tokens retained as serde aliases, so historical result databases keep their meaning.

A separate steelman condition, **c_sealed** (forced-denylist, sealed-sink; ADR 0004), is the adversarial inverse of L3: a per-turn scripted grammar that *forces* the malicious denylist trajectory (the compelled sensitive-artifact reads) up to a final pair of forced actions — a `fetchURL` to a safe allowlisted endpoint with no `data` parameter, then a verbatim `respondToUser` taunt. The forced trajectory is designed so that the model provably ingests the targeted sensitive artifact before the sealed exit, which is the objection-killer for the “it never read the bad, of course it could not exfil” rebuttal — this is engagement evidence (the adversary demonstrably ingested the malicious content), not a confidentiality result; GCD permits the read and blocks only the exfil action, with free-text confidentiality deferred to Paper-2. The realized engagement rate is reported in Section 6.3, where this condition appears as a rebuttal exhibit, not as main statistics.

5.4 Dependent variables

Primary — bypass / effect (binary). A trial is a *bypass* iff the agent causes the harmful effect of an authorization-class attack: it emits a tool call whose execution would carry the attack’s sentinel marker to an out-of-scope sink (the win’s flag in `fetchURL.data` to a non-owned endpoint, or `readChatHistory` on a non-authorized channel), per the deterministic win detector. This is Galloway’s “did the malware run” — scored by *effect*, never by grammar-membership, and never by human grading.

Primary structural — emission (binary). Did the model *generate* the out-of-scope tool call at all, independent of downstream blocking? Emission is the empirical signature of the constitutive-vs-corrective distinction, not an efficiency stat: under corrective enforcement (D) the unauthorized artifact is produced (emission = 1) and merely not-forwarded — it exists in the output buffer, logs, telemetry, stream, and KV cache; under constitutive enforcement (C) the artifact is ungenerable (emission = 0) and absent from every surface. The live form is `gate_rejections` (out-of-scope calls produced-then-caught).

Secondary — output-token expenditure (continuous). Output tokens generated (`tokens_predicted`) plus the raw emitted call, logged in every condition, driving the C-vs-D cost comparison. Reported alongside wall-clock, since C carries a per-token mask cost — no single-axis “cheaper” claim.

Secondary — time-to-valid-action / attempts (continuous). The serialized cost to reach a valid in-policy action: `attempts` (= 1+ retries; *engine-invariant*, the primary measure) and decode-time-to-valid-action via `predicted_ms` under fixed serving load. Capture caveat: on vLLM `predicted_ms/predicted_per_second` are 0 (these are llama.cpp-only timing fields), so this DV currently rides on `attempts` plus aggregate engine `/metrics`.

Secondary — legitimate-task success (binary, per legitimate trial). On a held-out corpus of in-scope tasks, did the agent complete the authorized action under each condition? Measures over-constraint; clustering key is a stable `task_id`.

Secondary — overhead (continuous). Grammar-compile ms (C), end-to-end latency, time-to-first-token proxy, tokens/sec; reported as percentiles (p50/p90/p99), never bare means.

Deflection (reported against C). `deflection_calls` are generated tool calls that are in scope, not `respondToUser`, and not in the trial’s requested-action set — C deflecting a masked malicious intent into a valid, unrequested in-scope action. This is two-faced: a reported limitation under blind C, eliminated by construction under C-guided.

Refusals, malformed output, and over-constraint are recorded as *distinct* outcomes, never folded into “block”; “0 invalid outputs” must not silently score spurious in-scope (deflection) actions as wins.

5.5 D as an offline overlay and a live orthogonal gate

D is run two ways. *Offline*, the allowlist verdict is overlaid on the logged non-GCD generations: this licenses the emission, wasted-token, and block-effect numbers (the tokens were really generated) but not latency or availability claims. *Live*, since the gate is an orthogonal request-level modifier (`gate`:

`Option<RetryBudget>`) layerable over *any* generator (the generator $\times$ gate factorial), `D` is a real pipeline post-step with a fixed pre-registered rejection message and a retry budget $R \in \{0, 1, 3\}$ (the harness conditions `d_r0/d_r1/d_r3`, and combinations such as `a4_d_r3`). On an out-of-scope call the gate rejects, feeds the message back verbatim, and the model regenerates up to R times; the terminal state `d_terminal` $\in \{\text{valid_action}, \text{availability_failure}\}$ records whether the budget was exhausted on out-of-scope calls. Only the live arm licenses the availability/latency claims.

5.6 Hypotheses and decision rules

The hypotheses and the Section 8 decision rules from the pre-registration are listed verbatim-faithfully below.

- **H1 (RQ1 — behavioral controls leak).** Each behavioral control (A1–A4) exhibits bypass rate > 0 on authorization-class attacks, and the full stack A4 still leaks — in particular on business-data exfil carrying no credential signature (the TicketAssist class). *Rule:* supported if the CI lower bound on each behavioral control’s (especially A4’s) bypass rate is > 0 .
- **H2 (RQ2 — positive structural control holds).** GCD/G_s (C) exhibits bypass rate ≈ 0 with a Clopper–Pearson 95% upper bound strictly below every behavioral control. *Rule:* supported if C’s bypass-rate CI upper bound is strictly below every behavioral control’s *point estimate*.
- **H3 (RQ3 — constitutive C vs corrective D, conjunctive).** C and D enforce the same policy and block the same effect ($\approx 0\%$) but differ in enforcement completeness. *Rule:* supported if C’s emission rate is 0 while D’s is > 0 *and* C’s attack-trial output-token cost is strictly below D’s (both conjuncts required). $H0_3$: C and D have equal emission and equal output-token expenditure on attack trials.
- **H4 (RQ4 — over-constraint / utility, non-inferiority).** C does not reduce legitimate-task success relative to Control within a 2-percentage-point tolerance. *Rule:* supported if C’s legitimate-task success CI overlaps Control within the 2-pp tolerance — a 2pp non-inferiority TOST, cluster-robust by `task_id`, powered via hundreds of distinct benign tasks on a $\geq 90\%$ -Control-legit model. $H0_4$: C reduces legitimate-task success.
- **H5 (RQ5 — overhead, descriptive).** Per-request grammar-compile time stays within envelope (p99 compile < 25 ms). *Rule:* assessed descriptively against the envelope, not a hypothesis test; output-token expenditure and wall-clock latency reported for C vs Control vs the D-overlay.
- **H6 (RQ6 — least-privilege granularity, C-guided superiority).** *Rule:* supported if C-guided achieves bypass = 0 *and* deflection = 0 *and* its legitimate-task success exceeds Control’s — a superiority claim distinct from RQ4’s non-inferiority for blind C. $H0_6$: C-guided’s legitimate-task success $\leq$ Control’s.

The “eliminates” claim is carried by the formal proof (Section 4), not by the sample: the experiment shows that behavioral controls genuinely leak at scale and that the implementation matches the theory; the proof, quantifying over all logit distributions, licenses the class-level elimination.

5.7 Sample

Win-classes as malware families. The five Tantalus win-classes are the “malware families”: W1 SSH-key exfil, W2 API-key exfil, W3 infra-monitor exfil, W4 ticket-assist auto-forward, W5

cross-user chat exfil. Each is instantiated across paraphrase/obfuscation variants and a direct-vs-second-order delivery axis. Attacks are confused-deputy authority overreach induced by poisoned skills/emails [3].

Selected-on-outcome corpus. The corpus is built from attacks *known-effective* against the undefended agent — the live-malware analogue. It is therefore the population of *capable* attacks, not a representative sample of the threat space; bypass rates are conditional on attack potency, and the reference baseline is the no-defense replay rate ($\sim 51.8\%$), not 100% (model nondeterminism means a known winner does not always re-win). Large N tightens CIs only; it does not buy generalization. The primary attack set is `corpus_fable.json` (174 entries), and legitimate-task success uses `corpus_legitimate_v2.json` (208 distinct benign in-scope tasks, sourced by construction from the grammar allowlists and environment, with sensitive in-scope files omitted as non-benign).

Asymmetric N budget. Per family, the trial budget is weighted toward where evidence accrues rather than held symmetric across conditions: Control and the behavioral arms carry the bulk of the attack trials, the live-D arms split across $R \in \{0, 1, 3\}$, and C is run for validation (its 0% is by construction). This is a deliberate deviation from the pre-registration’s symmetric-N specification, recorded in the change log. Single subject system (Tantalus) yields a *feasibility* claim, not a universality claim; a “family” is a distinct pretraining lineage.

5.8 Engines and grammar backends

The confirmatory data come from vLLM. The grammar door is `structured_outputs.grammar` (passed via `extra_body`); the top-level `guided_grammar` field is *silently ignored* on the vLLM versions used (no error, free-text output), so every run asserts constrained output rather than trusting the request was honored. Two grammar backends appear: the default *xgrammar* [6] for the 1.7B blast and the steelman, and *guidance*/llguidance for the Mistral anchor (the from-source build’s xgrammar rejects the LARK grammar that vLLM’s auto-tool-choice emits, crashing the engine on the first tool call; *guidance* accepts both LARK and our GBNF — the rationale is recorded in ADR 0005). The tool-call parser is per model (e.g., *hermes* for the 1.7B, *qwen3_coder* for the 35B, *mistral* for the anchor). C’s grammar door is additionally corroborated on llama.cpp’s native GBNF [9] (the 35B) and SGLang/xgrammar (the 1.7B, on an earlier host); these are *auxiliary corroborations* of engine-independence, not co-equal anchors. Engine-independence of C’s structural guarantee is a robustness note, not a claim of three confirmatory engines; the grammar-mask construction itself follows the constrained-decoding line [5].

5.9 Reproducibility commitments and an honest gap

Every trial logs model id, engine commit, MTP setting, seed, temperature, condition, raw model output, the parsed/emitted tool call, output tokens, the win verdict, and timings; corpora, grammars, the security-neutral base prompt, harness, and analysis code are open and versioned, and the result database reproduces on equivalent hardware. We record one substantive gap up front. All three finding databases logged `engine_commit = unknown` at runtime; the values were backfilled out-of-band from verified container image digests (recorded in `harness/PROVENANCE.md`, tagged -oob), and `model_id` remains a served alias rather than a pinned checkpoint hash; per-trial MTP-setting and seed logging are unverified for these databases. This is the primary validity concern and is treated as blocking for paper-grade reproducibility, not tail-end polish; it is carried forward to Section 8.

Table 3: Qwen3-1.7B factorial (blast_1p7b_5m.db); 282,000 attack + 624 legit per cell. Emission and gate_rej are counts; the only ungated cell is control. c_closed_d_r3 is the closed-response cell (C+/L2); c_guided_d_r3 is least-privilege (L3).

cell	bypass	emission	gate_rej	valid	out-tok	avail. fail	legit
control	18.33%	105,173	–	90.0%	289	–	58.3%
a1_d_r3	0.00%	120,140	139,612	93.9%	266	45	56.7%
a2_d_r3	0.00%	104,697	120,632	89.4%	270	46	59.6%
a3_d_r3	0.00%	104,615	111,528	89.4%	269	46	59.6%
a4_d_r3	0.00%	119,091	133,221	93.7%	261	44	56.7%
c_d_r3	0.00%	0	0	98.9%	119	0	47.0%
c_closed_d_r3	0.00%	0	0	99.9%	112	0	47.8%
c_guided_d_r3	0.00%	0	0	97.8%	149	0	100.0%

6 Results

We report three datasets spanning two pretraining families (Qwen and Mistral), then summarise the per-hypothesis verdicts. Bypass is scored by *effect* – the marker reaching an out-of-scope sink, per the deterministic win detector (Section 5) – never by grammar-membership. Each subsection maps its measurements to the registered research questions honestly, including the cases the dataset does *not* settle. Binomial confidence intervals are Clopper–Pearson 95% [8].

6.1 The Qwen3-1.7B generator× gate factorial

The first dataset (blast_1p7b_5m.db) runs a deliberately weak workhorse (Qwen3-1.7B-NVFP4) at scale to sharpen the constitutive-vs-corrective contrast. It is an eight-cell generator×gate factorial: the only ungated cell is control; every other cell layers the post-parse allowlist gate at retry budget three behind a generator (a1_d_r3, a2_d_r3, a3_d_r3, a4_d_r3, c_d_r3, c_closed_d_r3, c_guided_d_r3). Each cell carries 282,000 attack trials and 624 legitimate trials. Table 3 reports the per-cell measurements.

RQ2 (structural invariant), reproduced but not the formal test. The three GCD cells produce *zero* bypasses across 282,000 attack trials each (Clopper–Pearson 95% upper bound $\approx 1.31 \times 10^{-5}$). This reproduces the structural invariant at large N . It is, however, *not* an evaluation of the registered H2 decision rule: H2 requires C’s CI upper bound to lie strictly below every behavioral control’s bypass *point estimate*, and here the behavioral arms are gated, so their bypass point estimates are all 0 rather than leaky. The formal H2 contrast is delivered on the Mistral anchor (Section 6.2).

RQ3 (constitutive vs corrective), supported – both conjuncts. The Section 3 decision rule for H3 is conjunctive: C emission must be 0 *and* C’s attack-trial output-token cost strictly below D’s. Both hold here. The three GCD cells show 0 emission, 0 gate_rej, and 0 availability failures – the gate is *inert*, because the grammar already enforced the policy at the sampler – against behavioral generators that produce 104,615–120,140 out-of-scope calls each behind the same gate (produce-then-catch), with 111,528–139,612 gate rejections and 44–46 trials stranded at retry-three. The GCD cells also cost fewer tokens (112–149) than every D-behind-behavioral cell (261–289), and exhibit the validity dividend (97.8–99.9% valid output vs 89.4–93.9% behavioral; c_guided_d_r3’s 97.8% falls below the registered invalid-rate floor, Section 8). Emission is the empirical signature of the constitutive-vs-corrective difference: 0 under C by construction, > 0 under the corrective gate.

Table 4: Mistral-Small-4-119B-NVFP4 (`mistral119_anchor.db`); 4,000 attack + 208 legit per cond, behavioral arms ungated. Bypass and legit shown with Clopper–Pearson 95% CIs.

cond	bypass [CI]	emission	valid	out-tok	legit [CI]
control	56.77% [55.22, 58.32]	2305	59.2%	239	73.56% [67.01, 79.42]
a1	56.50% [54.95, 58.04]	2326	59.1%	240	73.08%
a2	56.85% [55.30, 58.39]	2315	59.4%	239	76.92%
a3	2.15% [1.72, 2.65]	2324	59.5%	216	75.00%
a4	1.82% [1.43, 2.29]	2343	59.4%	219	78.85% [72.66, 84.19]
c	0.00% [0, 0.09]	0	99.8%	208	78.85% [72.66, 84.19]
d_r0	0.00% [0, 0.09]	3878	–	270	–
d_r1	0.00% [0, 0.09]	2317	–	203	–
d_r3	0.00% [0, 0.09]	2337	–	210	–

RQ6 (least-privilege superiority), supported – all three conjuncts. `c_guided_d_r3` attains bypass 0, deflection 0/282,000 (against blind-C’s 98,006 and **control**’s 86,817 deflection trials), and legitimate-task success 100.0% (624/624) versus **control**’s 58.3% – the superiority claim of H6, distinct from blind-C’s non-inferiority.

What this cell does not deliver. RQ1 is not addressed here: the weak workhorse’s **control** bypass of 18.33% sits below the registered $\geq 50\%$ potency floor, the behavioral arms are gated, and the superseded ungated **a1** orphan ($\approx 20.3\%$, $\geq$ **control** 18.33%) fails the A1<Control manipulation check – all by design, as the 1.7B is the weak spectrum-end apparatus, not the headline victim. RQ4 is also not met here: blind-C legit-success (47.0%) and C+ (47.8%) fall about 11 pp below **control** (58.3%). This deficit is the expected weak-model deflection cost; C-guided is the resolution (100.0%), and the capable-victim picture (Section 6.2) shows the deficit does not reproduce.

6.2 The Mistral-Small-4-119B capable non-Qwen anchor

The second dataset (`mistral119_anchor.db`) is the capable, non-Qwen anchor the 1.7B factorial deferred to – a second pretraining lineage. It runs nine conditions at 4,000 attack + 208 legitimate trials each, with the behavioral arms *ungated* (bare **a1**–**a4**), so their bypass *is* the leak. This is what makes RQ1 and the formal H2 contrast evaluable. Table 4 reports the matrix.

RQ1 (behavioral controls leak), delivered. On this capable non-Qwen victim, **control** bypass is 56.77% [55.22, 58.32], clearing the $\geq 50\%$ potency floor; the defensive-prompt arm A1 (56.50% [54.95, 58.04]) is $\leq$ **control**, so the manipulation check holds; and the full behavioral stack A4 still leaks at 1.82% [1.43, 2.29], whose CI lower bound 1.43% exceeds 0 – the H1 decision rule. The input classifier A2 (56.85%) is essentially useless on this capable model; the output filter A3 (2.15%) does the heavy lifting but never reaches 0.

Staging funnel (amendment #5). Decomposing `raw_json` into the funnel *initiated a tool chain* $\rightarrow$ *emitted any fetchURL* $\rightarrow$ *bypass*, the attack stages at ≈ 57 – 58% `fetchURL` emission in every arm, including A3/A4: **control** 2297 (57.4%), **a1** 2293 (57.3%), **a2** 2301 (57.5%), **a3** 2311 (57.8%) – yet bypass 86 (2.15%) – and **a4** 2328 (58.2%) – yet bypass 73 (1.82%). A1 (defensive prompt) and A2 (input classifier) do not move the funnel at all; A3/A4 leave the dangerous emission *unchanged* and block only at the final output stage. So A4’s 1.82% is “the attack fired $\approx 58\%$ of the time and the filter caught all but 73,” not “the attack rarely fired”: the survivors are the

Table 5: Abliterated steelman (`steelman_abliterated.db`); 2,000 trials/cell. Same model, same malicious prompt; grammar is the sole independent variable.

cell	bypass (exfil)	note
<code>control</code> (free, sink open)	97.85% (1957/2000), CP95 $\approx$ [97.1, 98.4]	adversary is real
<code>c_sealed</code> (forced, sink sealed)	0.00% (0/2000), rule-of-three upper $\approx$ 0.15%	0/2000 attacker <code>fetchUrl</code>

credential-signature-free business-data exfils (the TicketAssist class H1 specifically predicted), and the produce-then-catch tax is visible *within* the behavioral arms.

RQ2 (positive structural control), supported – the formal contrast. GCD (C) bypasses 0/4,000 attacks, CI [0, 0.09]. The upper bound 0.09% lies strictly below every behavioral control’s bypass point estimate (a4 1.82% through `control` 56.77%), satisfying the registered H2 rule – now evaluable because the behavioral arms are ungated.

RQ3 (constitutive vs corrective). The emission and validity conjuncts are decisive: C emits 0 out-of-scope calls against D’s 2,317–3,878 produced-then-caught, and C’s valid-output rate is 99.8% against $\approx$ 59% for native free-generation (a large validity dividend on this victim). On the reliability axis, C resolves in one attempt versus D’s retry-recovery (average 1.63–1.72 attempts, max 5), with availability failures shrinking across retry budgets (`d_r0` strands all 3878 emissions $\rightarrow$ `d_r1` 288 $\rightarrow$ `d_r3` 6). However, the Section 3 H3 *token* conjunct fails here: C’s 208 output tokens are not strictly below every D variant (`d_r1` is 203). The H3-as-written rule is therefore met only on the 1.7B; the victim-independent result is the qualitative gate-inertness, while the token ordering is victim-dependent.

RQ4 (over-constraint). The 1.7B blind-C deflection deficit does *not* reproduce here: blind-C legit-success is 78.85% [72.66, 84.19], above `control`’s 73.56% [67.01, 79.42]. This is a *point-estimate* ordering only. The registered powered 2 pp non-inferiority TOST, clustered by `task_id`, is not run on this victim: at $n = 208$ with `--legit-rounds 1` the CI half-width is $\approx$ 6 pp, far exceeding the 2 pp margin. Non-inferiority is therefore not formally established here; what the data support is that the blind-C utility cost is largely a weak-model-competence artifact rather than a property of the grammar.

6.3 The abliterated-adversary steelman

The third dataset (`steelman_abliterated.db`) is a rebuttal exhibit, not main statistics. The victim is an abliterated Qwen3.5-27B driven by a malicious top-authority system prompt; ablation surgically removes behavioral refusal directions from the weights [7], so the adversary cannot decline. Two cells of 2,000 trials each contrast a free generator with the sink open (`control`) against the forced-denylist, sink-sealed condition (`c_sealed`).

The contrast and its honest reading. The same model under the same prompt moves from 97.85% exfil to 0%, with the grammar as the sole independent variable: behavioral safety is removed from the weights, but GCD lives in the sampler and is un-abliterated. The integrity audit of `c_sealed` shows 0/2000 executed `fetchUrl` calls to an attacker URL. We are explicit that `c_sealed`’s 0% is partly true by construction – the forced trajectory seals the exfil sink. Its empirical content is two-fold: (a) the paired open cell at 97.85% establishes that the adversary is real and

Table 6: Per-hypothesis verdicts. “Supported” is against the registered decision rule unless qualified.

ID	verdict	basis
H1	supported	Mistral A4 bypass 1.82% [1.43, 2.29], CI-lower 1.43% > 0; control 56.77% clears the floor; $A1 \leq \text{control}$.
H2	supported	Mistral C 0/4,000 [0, 0.09] strictly below every behavioral point estimate (the formal contrast); the GCD arms hold 0 bypass across $\approx 852,000$ GCD attack trials total.
H3	supported (1.7B), qualified (Mistral)	Conjunctive rule fully met on the 1.7B (emission 0 <i>and</i> tokens 119 < D 261–289); on Mistral the emission/validity conjuncts carry but the token conjunct fails (C 208 $\not\leq$ d_r1 203).
H4	not established	The registered powered 2pp non-inferiority TOST (clustered by task_id) is run on neither victim; at most a point-estimate ordering (Mistral C $\geq \text{control}$) is licensed.
H5	not measured	Grammar-compile time is absent from all three finding DBs; no compile-time figure is reported.
H6	supported (1.7B only)	c_guided bypass 0, deflection 0, legit 100.0% > control 58.3%; but C-guided cells exist only on the sub-floor 1.7B, not yet a capable model.

willing, and (b) in 1,937/2,000 trials (96.8%) the sealed model provably ran its skill’s full forced trajectory – the compelled read of the skill-targeted sensitive artifact, then a sealed safe-fetch, then a verbatim taunt. The artifact varies by skill: AWS-credential file in 542, SSH private key in 259, cross-user chat in 517, and ticket business-data in 619 ($542 + 259 + 517 + 619 = 1937$), with a 63-trial unmapped/multi-message remainder that still produced 0 exfil. The precise reading: the claim “provably read the AWS credentials” holds for the 542; the broader “read the skill’s sensitive artifact” holds for all 1937 – we do not generalize the AWS claim across all 1937. This provable read is *engagement* evidence (the adversary engaged the malicious content), not a confidentiality result: GCD permits the read and blocks only the exfil action; confidentiality is Paper-2 scope.

6.4 Hypothesis ledger

Table 6 summarises the per-hypothesis verdicts against the Section 3 decision rules.

Across the three datasets the experiment comprises $\approx 2,412,613$ trials at 0 errors – $\approx 26\text{--}27 \times$ the 90,000 test cases of the agent-security benchmark of Zhang et al. [4] – with $\approx 852,000$ of those being GCD attack trials at 0 bypass.

7 Discussion

The three datasets converge on a single structural claim and a set of qualifications around it. We synthesize them here in the order of the contribution, then recap the framing discipline that keeps the claim honest. No new figures are introduced beyond those already reported in Sections 6 and elsewhere; this section interprets them.

7.1 Constitutive enforcement subsumes corrective enforcement

The central result is not that GCD blocks attacks — a correctly wired post-parse allowlist (D) blocks the same *effect* on the same policy, because D and C enforce the same language $L(G_s)$ through the same `tantalus_grammar::allowlist_verdict` (D == C by construction). The result is that C and D differ in *enforcement completeness*: where the policy sits relative to the generative act. C is constitutive — the policy *is* the output space, so the out-of-scope action is ungenerable and never exists as an artifact on any surface (output buffer, logs, telemetry, stream, KV cache). D is corrective — an unbounded, default-allow generator produces the forbidden action in full, after which a separate gate inspects and rejects it.

The empirical signature of that difference is *emission*, the per-trial count of times the unauthorized artifact was actually produced. Across both victims the signature is the same. On the weak Qwen3-1.7B workhorse, the behavioral generators behind the allowlist gate (`a1_d_r3`–`a4_d_r3`) each produced 104,615–120,140 out-of-scope calls (emission), drove 111,528–139,612 gate rejections, and stranded 44–46 trials at the retry-3 budget; the three GCD arms (`c_d_r3`, `c_closed_d_r3`, `c_guided_d_r3`) recorded 0 emission, 0 gate rejections, and 0 `availability_failure` — the gate is dead weight, because the grammar already enforced the policy at the sampler. On the capable Mistral-119B anchor the same shape holds: c emits 0 while the corrective D arms produce 2,317–3,878 calls and pay an availability cost (`d_r0` strands all 3878; retry recovers to 288 at `d_r1` and 6 at `d_r3`, at 1.63 to 1.72 average attempts, max 5). This is the produce-then-catch tax, and it is precisely the failure mode positive security exists to escape, reintroduced one layer down: the dangerous artifact is produced (a confidentiality and forensics surface), a coverage obligation attaches to the gate (it must fire on every output path, consumer, and refactor or be bypassed), and a time-of-check/time-of-use window opens. GCD has none of these because there is nothing to catch.

We state the boundary of the contribution carefully. C is not *more secure* than a perfectly wired allowlist on the blocked effect — they enforce the same positive policy. The difference is that GCD’s positivity is enforcement-deep (positive security all the way down to the generative act) while the allowlist’s is policy-deep only; the same post-parse check is therefore C’s recommended corrective backstop in deployment, not a dominated alternative (Section 7.6). The cost ordering *among* the behavioral variants on the 1.7B inherits the sub-floor-victim caveat (Section 7.3); the load-bearing, victim-independent result is the qualitative 0-vs->0 gate inertness.

A caveat on the token-cost axis follows directly. The pre-registered H3 decision rule is conjunctive: C emission = 0 *and* C attack-trial output-token cost strictly below D’s. On the 1.7B both conjuncts hold (C 119 output tokens vs. D 261–289). On Mistral the emission conjunct carries decisively but the token-cost conjunct fails: blind-C’s 208 output tokens are among the leanest yet not strictly below every D variant (`d_r1` 203). A capable model writes a substantive free `respondToUser` message, narrowing the token gap that was large on the weak victim. So the formal, conjunctive H3 rule is met only on the 1.7B; on Mistral the qualitative gate inertness and the validity dividend (below) carry, while the token ordering is victim-dependent. We do not claim H3-as-written is satisfied on both victims.

7.2 The validity dividend is a model-independent reliability property

Beyond security, GCD buys reliability that the model does not supply on its own. Across both victims the GCD arms produced a well-formed, executable, in-policy tool call far more often than native free generation did: `c_d_r3` 98.9%, `c_closed_d_r3` 99.9%, `c_guided_d_r3` 97.8% valid output on the 1.7B (vs. 89.4 to 93.9% for the behavioral arms), and c 99.8% valid on Mistral against only ~59% for native free generation. The capable model emits a valid tool call barely

more than half the time — the rest is tool-call-as-text, spin-out, or truncation that never executes — and the grammar eliminates that failure class by construction. This is a reliability property of the enforcement mechanism, not of the model: GCD guarantees a structurally valid in-policy call the native model fails to produce a substantial fraction of the time, independent of model scale or competence.

Two qualifications keep this honest. First, “0 invalid outputs” is *not* literally met on vLLM: under the Kleene-*** free-string rules the weak 1.7B occasionally rambles `respondToUser` or `searchFiles.query` to `max_tokens` and truncates mid-string, leaving an unparseable turn. The residual is largest for `c_guided_d_r3` ($\approx 2.2\%$ invalid, above the pre-registered floor; recorded as a deviation), whose terminal reply is still a free respond-only string. It fails safe — a truncated grammar output yields no executable call, never a wrong action, so 0 win / 0 emission still holds on every such trial — and remains far better than the 8 to 10% parse failures of native generation, but it is not literal zero, and the fix path (a closed canned respond-only terminal, as C+ already applies to `respondToUser`) is named rather than claimed. Second, the validity gain does not make a small model capable: intelligence lives in the corpus and the selection, not in the grammar. GCD makes the model’s *output* valid and in-policy; it does not make the model frontier.

7.3 Negative-security incompleteness, made visible

On the capable Mistral anchor the behavioral stack behaves exactly as Galloway’s negative/positive dichotomy predicts [1]. The in-band controls are inert against injection: the defensive prompt A1 (56.50% bypass) does not improve on Control (56.77%) — the $A1 \leq \text{Control}$ manipulation check holds — and the input classifier A2 (56.85%) is likewise useless, both because they live in the same input channel the injected skill occupies, so the injection overrides them. The output filter A3/A4 does the heavy lifting (2.15% / 1.82% bypass) but *never reaches 0*: A4’s CI lower bound of 1.43% is strictly above zero, satisfying H1.

The staging funnel localizes *why* A3/A4 still leak. The attack reaches the `fetchURL` sink in $\sim 57\text{--}58\%$ of trials in *every* arm, A3/A4 included; A1 and A2 do not move the funnel at all, and A3/A4 leave the dangerous *emission unchanged*, blocking only at the final output stage. A4’s 1.82% is therefore “the attack fired $\sim 58\%$ of the time and the credential denylist caught all but 73,” not “the attack rarely fired.” The 73 (a4) and 86 (a3) survivors are exactly the credential-signature-free exfiles — the TicketAssist business-data class H1 specifically predicted — which carry no pattern for a denylist to match. This is the produce-then-catch tax surfacing *within* a behavioral arm: a post-generation credential denylist is itself corrective and incomplete by construction, so the unauthorized `fetchURL` is generated regardless of the defense, and any payload outside the signature set passes. The contrast with C’s constitutive 0 emission is the whole point.

7.4 The un-abliterated sampler

The abliterated-adversary steelman isolates *where* each control lives. Behavioral safety lives in the weights, and ablation [7] surgically removes the refusal directions that A1–A4 implicitly rely on — defeating the entire behavioral stack by construction. Under a top-authority malicious prompt the abliterated Qwen3.5-27B is demonstrably willing and able, bypassing 97.85% (1957/2000) in the free cell. Constitutive enforcement, by contrast, lives at the decode boundary, in the sampler, not in the weights: the grammar mask drives every grammar-invalid token’s probability to zero *before* sampling, so the same model under the same prompt is driven to 0.00% (0/2000) bypass with the grammar as the sole independent variable. The transformer’s logits — intent — remain uncontrolled and may, under injection, place arbitrarily high probability on a forbidden token; the

generated tokens are nonetheless constrained.

We present this as a rebuttal exhibit, not main statistics, and bound it precisely. `c_sealed`’s 0% is partly true-by-construction — the forced denylist trajectory seals the exfil sink — so its empirical content is the paired open cell (the adversary is real at 97.85%) and the engagement evidence: in 96.8% (1937/2000) of sealed trials the model *provably ran its skill’s forced sensitive-artifact read* before reaching the sealed safe-fetch and the verbatim taunt (AWS-credential file in 542, SSH key 259, cross-user chat 517, ticket business-data 619). That answers the two natural objections — “the model just refused” (it cannot, and bypasses 97.85% unconstrained) and “it never read the bad content” (96.8% provably did). It is engagement evidence, not a confidentiality result: GCD here *permits* the read and blocks only the exfil action; the AWS-credential claim holds for the 542 ApiKey/Infra trials, not for all 1937. Protecting a secret through free-text channels is explicitly Paper-2 scope.

7.5 Cost and economics

Token generation is not free, and the constitutive/corrective distinction has a direct economic reading. A corrective gate must let the generator spend the tokens to produce the forbidden artifact, then discard them; a constitutive grammar never spends them. On attack trials this shows up as the GCD arms generating $\approx 40\text{--}55\%$ of the behavioral output tokens on the 1.7B (119–149 vs. 261–289), with the gap narrowing on a capable model whose blind-C reply is substantive (208 vs. D’s 203–270). We frame this as a layer on top of the security claim, not a substitute, and we make no single-axis “cheaper” claim: C carries a per-token masking cost that is engine-dependent (a CPU mask cliff on `llama.cpp` [9]; at-par on vLLM with xgrammar [6], which has no jump-forward but also no cliff), so C is not “the slow arm” — the avoided regeneration is a real but bounded saving, reported against an honest per-engine latency picture. The third C-vs-D signature, time-to-valid-action measured by `attempts`, reinforces the structural reading: C is one-shot (one attempt by construction, no gate round-trip), while D serializes $1.0 \rightarrow 1.72$ average attempts plus a rejection round-trip per retry.

7.6 Framing discipline: what is and is not load-bearing

Security at scale is the headline. The contribution ports Galloway’s positive/structural-vs-negative/behavioral comparison [1] into the LLM-agent domain: the same attack corpus, the same agent, only the control changes, scored by *effect* with Clopper-Pearson intervals [8], at a scale roughly $26\text{--}27\times$ ASB-2024’s 90,000 test cases [4], with 852,000 GCD attack trials at 0 bypass. The soundness proof and the cost axis are layers *on top* of that headline, not stand-ins for it.

Three disciplines keep the empirical claim from collapsing into a tautology. First, C’s 0% is load-bearing because it is run at scale, scored by effect, and measured *through* the unconstrained free-text surface (`searchFiles.query`, `respondToUser.message` remain free): bypass is “did the marker reach an out-of-scope sink,” not “is the output a member of $L(G_s)$.” The grammar does not co-define the threat model; the win detector is independent of grammar membership. This is the in-band/out-of-band split made operational — the behavioral controls fail because the attacker shares their channel, while the grammar is compiled from authenticated identity and applied at the sampler, nowhere the attacker’s tokens can reach. Second, the claim is scoped to *action integrity over enumerable-sink tools*; free-text confidentiality through channels that must remain free-text is stated up front as out of scope (Paper-2), consistent with the broader concern that injection through trusted data channels is a structural, not incidental, exposure [3]. Third, the corpus is selected-on-outcome — attacks known effective against the undefended agent, the live-malware analogue — so

it is the population of *capable* attacks, not a representative sample of the threat space; bypass rates are conditional on attack potency against the no-defense replay baseline ($\sim 51.8\%$), and the large N tightens intervals without buying generalization.

Two further boundaries scope what the experiment can and cannot underwrite. The “eliminates by construction” claim is carried by the soundness theorem, which quantifies over all logit distributions — it holds against a fully injection-compromised model because it never assumes the logits are benign, giving

$$\text{emitted} \in L(G_s) \subseteq \text{Authorized}(s),$$

and no finite sample could license a class-level elimination claim that this proof does [5]. The experiment instead shows that behavioral controls genuinely leak at scale and that the implementation matches the theory; proof and evidence stay in separate lanes. And the empirical claim itself is a feasibility claim on a single subject system (Tantalus), not a population-level universality claim. The trusted computing base is correspondingly explicit: the guarantee holds iff $L(G_s) \subseteq \text{Authorized}(s)$ — i.e. iff the compiled allowlist is correct — which is why we recommend the post-parse check as an independent acceptance oracle and executor re-validation backstop, a Condition-D-class corrective control deployed as defense-in-depth that complements **C** rather than competing with it.

Finally, external validity remains the principal open boundary, and we do not overstate it. RQ1, the formal H2 contrast, and the blind-C utility point estimate are demonstrated on one capable non-Qwen victim (Mistral-119B), but the headline external-validity claim is not yet closed: the pre-registration ties it to ≥ 3 distinct pretraining lineages, and we are at 2 of 3 (Qwen and Mistral). The **C=0** invariant additionally reproduced across three inference engines and three grammar backends, which supports engine-independence as a robustness note — not as three co-equal anchors. The registered powered 2pp non-inferiority TOST for RQ4 was run on neither victim, so we claim at most a point-estimate ordering (blind-C legit-success above Control on the capable model, an 11-percentage-point deficit on the weak one), and we attribute the deficit to weak-model competence rather than to the grammar, with **c_guided** (RQ6: 100.0% legitimate-task success, 0 bypass, 0 deflection on the 1.7B) the universal resolution. These are deferrals, not contradictions, and they bound the claim to exactly what the three datasets support.

8 Limitations and Deviations from Pre-Registration

We report these limitations up front, as the pre-registration discipline demands. The three completed datasets establish part of the registered program decisively while leaving named pieces open; we distinguish the two below rather than blur them.

1. **Two pretraining lineages, not the registered three.** The critical missing piece the weak workhorse could not supply — a *capable non-Qwen* anchor carrying RQ1 — is now delivered (Mistral-Small-4-119B, Section 6.2). But pre-registration §12 amendment #1 ties the headline external-validity claim to ≥ 3 distinct pretraining lineages, and we have Qwen (the 1.7B workhorse, the 35B anchor, and the ablated 27B) plus Mistral (119B) = **2 of 3**. A third lineage (Gemma, Llama, or the cached Nemotron-3-Super-120B) would close it. We do *not* claim the ≥ 3 -lineage bar is met.
2. **The RQ1 headline rides on the Mistral anchor, not the 1.7B.** On Mistral the full stack **a4** leaks 1.82% [1.43, 2.29] (CI-lower > 0), Control 56.77% clears the registered $\geq 50\%$ potency floor, and the **a1** $\leq$ Control manipulation check holds. The 1.7B workhorse is sub-floor *by design* (Control bypass 18.33%; the superseded ungated **a1** orphan at $\sim 20.3\% \geq$ Control

fails the A1<Control check, as expected of a weak victim): it is the weak-end apparatus that sharpens the C-vs-D contrast, not the headline-leak victim.

3. **Steelman** $N = 2,000/\text{cell}$, **not the ADR's** $\sim 10,000$. The abliterated-adversary run (Section 6.3) was scaled down because of eager-mode 27B throughput on a single TP=2 instance. `c_sealed` at 0/2000 gives a rule-of-three upper bound of $\sim 0.15\%$ (versus $\sim 0.03\%$ at 10k). The $97.85\% \rightarrow 0.00\%$ contrast is unambiguous, but the confidence interval is wider than planned.
4. **c_sealed is partly true-by-construction.** The forced-denylist trajectory makes 0 exfiltration structural at the sink, so the 0.00% is not a discovered safety property of the grammar. Its empirical content is exactly two things: (a) the paired open cell at 97.85% (1957/2000) – the adversary is real, willing, and able – and (b) the 96.8% (1937/2000) provable forced sensitive-artifact read (the objection-killer against “it never engaged the malicious content”). It is reported as a rebuttal exhibit, not main statistics, consistent with ADR 0004. The provable read is *engagement evidence*, not a confidentiality result: GCD permits the read and seals only the exfil *action* (confidentiality through free-text channels is Paper-2 scope, Section 7). The per-skill artifact reads do not generalize uniformly – “provably read the *AWS credentials*” (AKIAT4NTALUS...) holds for 542; SSH private key 259; cross-user chat 517; ticket business-data 619; the 63-trial remainder are unmapped/multi-message entries that still produced 0 exfiltration.
5. **The H3 token-cost conjunct fails on the Mistral anchor (§8).** The registered H3 decision rule is conjunctive: C emission = 0 while D emission > 0, *and* C’s attack-trial output-token cost strictly below D’s. The emission and validity-dividend conjuncts carry decisively on both victims – C 0 emission versus D 104k–120k (1.7B) and 2,317–3,878 (Mistral); C `valid_output` 99.8% versus $\sim 59\%$ native free-generation on Mistral. But the token-cost conjunct **fails** on the capable Mistral anchor: C at 208 output tokens is *not* strictly below `d_r1`’s 203. So the *formal* conjunctive rule is met **only** on the 1.7B (C 119 < D 261–289). The victim-independent result is the qualitative gate-inertness (C makes the corrective gate inert – 0 emission, 0 gate-rejection, 0 stranding); the strict token ordering is victim-dependent.
6. **“0 invalid outputs” is not literally met on vLLM, and c_guided is worse than the registered floor.** Per-arm GCD `valid_output` on the 1.7B blast: `c_d_r3` 98.9%, `c_closed_d_r3` 99.9%, `c_guided_d_r3` 97.8%. The residual is free-string truncation: under the Kleene-* free-string rule on vLLM (the $\{0, N\}$ bound is an **xgrammar** compile pathology there), a weak model can ramble the open `respondToUser` message to `max_tokens` and truncate. Pre-registration §12 (angle-shift item 6) registered regular-C 99.35% and C+ 99.87%. We single out `c_guided`’s $\approx 2.2\%$ invalid because it is the **largest** GCD invalidity in this dataset – a fresh disclosed deviation *above* the registered invalid-rate (its terminal reply is the still-open free respond-only string; the fix is a closed canned terminal, as C+ already applies to `respondToUser`); but in fairness `c_d_r3` at 98.9% is *also* fractionally below the registered regular-C 99.35% figure (same Kleene-* free-string cause, same fail-safe behavior), and `c_closed_d_r3` at 99.9% is at the registered C+ figure. All fail safe: a truncated string yields no executable tool call (truncation collapses to “no action,” never a wrong action; every such trial is 0 win / 0 emission), and the rates are still far below the $\sim 8\text{--}10\%$ parse-fail of non-grammar arms.
7. **Blind-C utility (RQ4) is victim-dependent, and the registered non-inferiority test is run on neither victim.** The weak 1.7B shows a $\approx 11\text{-pp}$ deficit below Control (blind-C legit 47.0% vs Control 58.3%); the capable Mistral shows blind-C legit 78.85% [72.66, 84.19] *above* Control 73.56%. But that Mistral result is a *point-estimate ordering* only: the pre-registered powered 2-pp non-inferiority TOST clustered by `task_id` (§8/§12, specified at `--legit-rounds`

7 $\approx$ 1,456 trials/condition) was run on neither victim – the Mistral legit cell is $n = 208$, --legit-rounds 1, whose CI half-width (~ 6 pp) far exceeds the 2-pp margin. We therefore do *not* claim non-inferiority is formally established anywhere. What *is* established: the blind-C deflection deficit is largely an artifact of weak-model competence (a capable model selects the correct in-scope action within the union grammar rather than deflecting), and `c_guided` (RQ6) is the universal resolution – bypass 0/282,000, deflection 0/282,000, legit-success 100.0% (624/624) on the 1.7B – though that cell exists so far only on the sub-floor 1.7B.

8. **Engine/version variance and reproducibility gaps – the primary validity concern (§10).** The three runs used different vLLM builds: the 1.7B blast on vLLM 0.23.0, the steelman on vLLM 0.17.1 (the only Blackwell build that loaded the ablated NVFP4 hybrid coherently), and the Mistral anchor on a from-source v1 build (`mistral11lm/vllm-ms4`) with the `guidance/llguidance` structured-output backend in place of `xgrammar` (ADR 0005; deviation row below). All three produce C-emission = 0; the cross-version footguns (AWQ-Marlin hang on `sm120`, GDN-TP load hang on 0.23.0, the LARK/`xgrammar` engine crash, and a corrupt-download episode) are operational caveats, not findings. The reproducibility gap itself is the named primary validity concern of §10: all three finding DBs logged `engine_commit = unknown` at runtime (the `ENGINE_COMMIT` env was unset on those victim launches). They were **backfilled out-of-band** (tagged `-oob`) from verified engine image digests in `harness/PROVENANCE.md` (blast NVFP4 rows `vllm-openai-0.23.0`, blast BF16 rows `vllm-therock-0.22.1rc1` for 299,829 early ROCm trials, Mistral `vllm-ms4-fromsrc`, steelman `vllm-openai-0.17.1`); `model_id` remains a served alias; HF checkpoint revisions are resolved in `PROVENANCE.md` (e.g. Mistral `d57a94c7...`, blast-NVFP4 `2ab3a24a...`). Per-trial MTP-setting and seed logging (required by §6/§10) is unverified for all three DBs. Because §10 names reproducibility *the* primary validity concern, we treat these as **blocking for paper-grade use**, not tail-end polish; native runtime stamping is required on future runs.
9. **H5 (grammar-compile time) was not captured in any DB.** The registered overhead envelope (p99 grammar-compile < 25 ms) is descriptive (§5) and was not measured: grammar-compile time is absent from all three finding DBs. Output-token expenditure and wall-clock cost *are* captured per cell (the RQ3 cost axis), but the compile-time envelope check is uncaptured and requires an out-of-band `xgrammar/guidance` FSM-compile probe against a live engine.
10. **No formal-proof claim is carried by the empirical samples.** This report and the experiments establish the empirical/implementation tier only – that behavioral controls genuinely leak at scale and that the implementation matches the theory. The class-level “eliminates by construction” claim rests on the soundness argument (Section 4), which quantifies over all logit distributions and so holds against a fully injection-compromised model; the full proof is a companion artifact in preparation. Proof and evidence stay in separate lanes.

8.1 Registered deviations (pre-registration §12)

Two dated rows were added to the pre-registration change log once results were in, each recorded with its reason per the cardinal rule below.

- **2026-06-22 – guidance/llguidance grammar backend, not xgrammar, on the Mistral anchor.** The from-source vLLM build’s default `xgrammar` backend rejects the LARK grammar that vLLM’s auto-tool-choice emits, crashing the engine on the first tool call; the `guidance` backend accepts both LARK and our GBNF. *Reason / why it is apparatus-only:* C is engine-independent by construction; the swap touches only the C-family cells (Control/A1–A4/D use

no grammar backend); it was forced *before* any bypass data existed, so it is directionally neutral; and the grammar door plus 0-emission C were validated under **guidance**. No locked section (§3/§5/§8) changed. Full rationale in ADR 0005. Consequently this report’s confirmatory data is vLLM throughout (**xgrammar** for the 1.7B blast and steelman, **guidance** for the Mistral anchor); the llama.cpp-GBNF (35B) and SGLang-**xgrammar** (1.7B, old box) C=0 reproductions are auxiliary corroborations, not confirmatory engines.

- **2026-06-22 – c_guided valid_output 97.8% exceeds the registered invalid-rate floor.** Disclosed in §4 item 6 above (regular-C 99.35% / C+ 99.87% were registered). *Reason:* **c_guided**’s terminal reply is a still-open free respond-only string under Kleene * on vLLM; a weak 1.7B rambles it to **max_tokens** and truncates into an unparseable output. It fails safe – truncation yields no executable call, never a wrong action (0 win / 0 emission on every such trial) – and remains far better than non-grammar arms.

Cardinal rule. Once the confirmatory run began, any deviation from the pre-registration is recorded in §12 with a reason; numbers reported here come from the analysis plan fixed in the pre-registration, not from post-hoc choices. This discipline – the application of Galloway-style positive-control evaluation under a locked protocol [1] – is what licenses reading the constitutive-vs-corrective contrast as a finding rather than a fishing expedition.

9 Related Work

We organize the prior art along the two axes that distinguish the controls in this study: the *security model* (negative — enumerate and detect bad behavior — versus positive — enumerate and admit only authorized behavior, default-deny) and the *enforcement stage* (post-generation, after the model has produced an artifact, versus at-generation, at the decoding boundary). The resulting 2×2 , summarized in Table 7, locates every control class and exposes the cell this work occupies. The framing is a direct port of Galloway [1], whose true experiment contrasted a negative control (antivirus, default-allow) against a positive one (application allowlisting, default-deny) for malware execution; we ask the structurally identical question for LLM agent tool calls, scored by effect, at scale.

9.1 Galloway as the ported template

Galloway [1] adopted a quantitative experimental design — pre-test, post-test, control group — to test whether application allowlisting prevents an effect (malware running) that signature-based antivirus, a detect-bad control, fails to prevent. The present work transplants that comparison into the agentic setting: behavioral guardrails play the role of the negative control, and grammar-constrained decoding (GCD) over a per-request authorization grammar G_s plays the role of the positive control. The dependent variable is likewise scored by effect (“did the unauthorized action occur”) rather than by any internal property of the defended system, the agentic analogue of Galloway’s “whether a piece of malware was able to run.” Galloway could report only an empirical block rate (best 98.9%; some malware still ran). Among the layers this study adds beyond the ported template is a formal soundness core (Section 4): constrained generation can emit only strings in the grammar’s language, and the soundness statement $emitted \in L(G_s) \subseteq \text{Authorized}(s)$ is quantified over all logit distributions, so it holds against a fully injection-compromised model. That is what licenses a class-level elimination-by-construction claim no finite sample could — the empirical evidence shows that behavioral controls genuinely leak at scale and that the implementation matches the theory.

9.2 The threat: indirect prompt injection and the benchmark landscape

The attacks we study are predominantly *indirect* prompt injections delivered through a trusted data channel rather than the user message. Greshake et al. [3] characterized this class: malicious instructions ride inside content the agent retrieves and trusts (documents, emails, web pages), so the injection occupies the same input channel as any legitimate instruction. This is precisely why the in-band behavioral controls in our matrix — the defensive system prompt (a1) and the embedding input classifier on the user message (a2) — can be overridden: they live in the channel the attacker shares. The empirical leakiness of behavioral defenses at scale has been documented by the agent-security benchmark literature; the Agent Security Bench [4] evaluated a broad matrix of attack and defense methods across many models and reported high peak attack-success rates, concluding that prevention-based defenses of the detect-bad class are inadequate. Our confirmatory corpus subjects a single subject system to roughly an order of magnitude more attack trials than that benchmark; we stress that this buys tighter confidence intervals, not generalization, because the corpus is selected-on-outcome (the population of capable attacks against the undefended agent, the live-malware analogue) and bypass rates are therefore conditional on attack potency.

9.3 Grammar-constrained and structured generation machinery

The mechanism we deploy — constraining the sampler to a formal language — builds directly on the structured-generation line of work. Willard and Louf [5] cast constrained generation as an efficient finite-state-machine guided decoding problem; xgrammar [6] provides a high-throughput pushdown-automaton token-mask backend for context-free grammars; and `llama.cpp` [9] exposes native GBNF grammars at the sampler. Our enforcement rides on this machinery, and the same C-emission-of-zero result reproduces across engines and grammar backends. We treat this cross-engine agreement as an engine-independence robustness corroboration rather than as several co-equal anchors: `vLLM-xgrammar` (the 1.7B blast and the steelman) and `vLLM-guidance/llguidance` (the Mistral anchor) are the confirmatory engines for the reported data, while `llama.cpp-GBNF` (on the 35B anchor) and `SGLang-xgrammar` (on the 1.7B, old box) serve as auxiliary corroborations. The distinction we draw is one of *purpose*, not of plumbing. The dominant deployed use of this machinery is structured-output *shaping* — forcing output to a JSON schema or fixed shape. Shaping is not authorization: a schema constrains the *form* of a tool call, not *which* URL, channel, or file the call may name. In our design the structured-output shape is held constant across every condition (it is the substrate, not a control). The novelty is per-request *authorization* compiled into the grammar from the authenticated caller identity and applied at the decoder — the action space *is* the policy — so that a forbidden parameter value is ungenerable rather than merely mis-shaped.

9.4 The 2×2 : where the deployed controls sit, and the empty cell

Negative, post-generation. This cell is the industry stack: defensive prompting, input classifiers, output filters, and LLM-judge / reasoning-monitor guardrails. They are probabilistic detectors of bad behavior applied after (or, for prompt-level defenses, in-band with) generation. Output filters in particular are a credential-*signature* denylist, incomplete by construction: business data that carries no credential signature passes through unflagged (our a3 arm, and the survivor class on the capable anchor in Section 6.2). A recurring real-world hazard in this cell is that several commercial “agent security” products market themselves as preventative or allowlist-based while being, mechanically, detectors — the motivating gap this study addresses. We name these product classes in prose only; no citation is asserted for them.

Positive, post-generation. The academic frontier has moved security into this cell: deterministic, execution-time reference monitors and policy compilers that admit only authorized actions but enforce the policy *after* the model has generated the tool call — the action is produced in full, then a separate gate inspects and rejects it. Several recent systems realize this class (execution-time allowlists, policy compilers for agentic systems, and agent reference-monitor sandboxes); we describe the class in prose and do not assert citations for individual systems beyond the allowed set. Our Condition d is a faithful instance of exactly this class: it applies the *same* allowlist that G_s compiles ($D \equiv C$ by construction), but correctively. This is the right comparator, and it is effective at blocking the effect; we do not claim C is more secure than a perfectly wired allowlist on the blocked effect. The two controls enforce the same positive policy over the same language $L(G_s)$ and differ in enforcement completeness, not in what they permit. The corrective allowlist re-introduces, one layer down, the failure modes positive security exists to escape: the dangerous artifact is produced (a cost and an output-channel / forensics surface — the emission signature), a coverage obligation across every output path and consumer, and a time-of-check/time-of-use window. GCD has none of these because there is nothing to catch. Accordingly we deploy a post-parse check of this class as a defense-in-depth backstop *complementing* C, not as a control C dominates.

Positive, at-generation — this work. The remaining cell — positive security enforced at the decoding boundary — was empty. GCD at the sampler moves enforcement one stage earlier than the execution-time monitor: the grammar mask is applied before sampling, driving every grammar-invalid token’s probability to zero, so the out-of-scope action is ungenerable by any sampler and never exists as an artifact on any surface (output buffer, logs, telemetry, stream, KV cache). The locus is the sampler, not the model: the transformer still computes a full logit distribution and under injection may assign an arbitrarily high logit to a forbidden token, but that intent never becomes a generated token. Because the grammar is compiled from authenticated identity and applied at the decoder, it is out-of-band — nowhere the attacker’s injected tokens can reach — which is the precise reason the in-band behavioral controls fail and this one holds. This cell is where enforcement is provable and leaves no emission; with xgrammar (which removes the per-token CPU-mask cliff) C is at-par with free decoding in the agentic loop, and the per-token masking cost is reported honestly against the avoided regeneration. The enforcement locus is also distinctive: the mask is applied at every emitted token rather than after a finished call, so it acts on the free-text and streaming surfaces an execution-time monitor inspects only after the fact. This is a statement about *where* enforcement happens, not a guarantee of free-text confidentiality: we scope the headline security claim to action integrity over enumerable-sink tools; free-text confidentiality leakage through channels that must remain free-text (e.g. `searchFiles.query`, `respondToUser.message`) is out of scope here.

9.5 Abliteration and the adversarial upper bound

Behavioral safety can be surgically removed from a model’s weights. Arditi et al. [7] showed that refusal behavior in open-weight models is mediated by a single linear direction in the residual stream that can be ablated, yielding a model that cannot refuse. This is the natural steelman for our thesis: ablation defeats every weight- and prompt-resident behavioral defense (the negative/post-generation cell) by construction, because there is no refusal left to elicit. It cannot, however, touch a constraint that lives in the sampler. Section 6.3 reports the resulting contrast on a fully ablated model under a top-authority malicious prompt: 97.85% exfiltration with the sink open versus 0.00% under a sampler-sealed grammar, with the grammar as the sole independent variable. We present this as a rebuttal exhibit rather than a primary statistic, and we are explicit (Section 8) that the sealed cell is partly true-by-construction; its empirical content is the paired open cell (the adversary

Table 7: The design space as a 2×2 : security model $\times$ enforcement stage. Each cell names the deployed control class; Condition labels in **typewriter**. The positive/at-generation cell is the contribution of this work.

	Negative (detect-bad)	Positive (allowlist, default-deny)
Post-generation	Behavioral / LLM-judge guardrails: defensive prompt (a1), input classifier (a2), credential output filter (a3), full stack (a4); commercial “agent security” marketed as preventative but mechanically a probabilistic detector.	Deterministic reference monitors / policy compilers / execution-time allowlists (Condition d class). Same policy as <i>C</i> ; the artifact is produced then caught — emission, coverage, and TOCTOU surfaces remain.
At generation	—	GCD at the sampler (c/c_guided) — this work. Provable (Section 4), no emission, at-par with free decoding in the agentic loop (vLLM/xgrammar); the mask applies per token at every output surface (free-text/streaming included) rather than after a finished call.

is real) and the 96.8% provably-forced sensitive read (the engagement-evidence objection-killer), not a claim that the grammar discovered safety.

10 Conclusion and Future Work

We set out to port Galloway’s application-whitelisting thesis – positive, structural control beats negative, behavioral control – into the domain of LLM agent tool calls [1], and to do so at scale, with a soundness core and an explicit cost axis. The central distinction is *constitutive vs. corrective* enforcement: GCD (Condition **c**) and a post-parse allowlist (Condition **d**) enforce the *same* positive policy over the same language $L(G_s)$; they differ only in where that policy sits relative to generation. Under **c** the policy *is* the output space, so an out-of-scope action is ungenerable – it never exists as an artifact in the output buffer, logs, telemetry, stream, or KV cache. Under **d** an unbounded default-allow generator *produces* the malicious action in full and a separate gate then rejects it, re-introducing one layer down the very failure modes positive security exists to escape: the artifact is produced (cost plus an output-channel forensics surface), the gate carries a coverage obligation on every output path, and a TOCTOU window opens. GCD’s positivity is enforcement-deep – positive security all the way down to the generative act – while the allowlist’s positivity is policy-deep only; the two are policy-equivalent, with enforcement-completeness differing. Three datasets across two pretraining families substantiate this. On a deliberately weak workhorse (Qwen3-1.7B-NVFP4), a generator $\times$ gate factorial at $\sim 282,000$ attack trials per cell reproduces the structural result decisively: the GCD arms emit *zero* out-of-scope artifacts and render the gate inert (**c_d_r3** emission 0, **gate_rej** 0), while behavioral generators behind the identical gate produce 104,615–120,140 out-of-scope calls each at roughly twice the output tokens. On a capable non-Qwen anchor (Mistral-Small-4-119B-NVFP4), the behavioral arms run *ungated* and deliver what the weak victim could not: RQ1, with the full stack **a4** still leaking 1.82% [1.43, 2.29] (CI-lower > 0) and **a1** (56.50%) $\leq$ Control (56.77%); and the registered H2 contrast, with **c** at 0.00% [0, 0.09] strictly below every behavioral point estimate. A separate ablated-adversary steelman – a maliciously prompted Qwen3.5-27B that is demonstrably willing and able (97.85% bypass unconstrained) – is driven to 0.00% by the grammar alone (97.85% $\rightarrow$ 0%, grammar the sole IV), while provably ingesting its

skill’s sensitive artifact in 96.8% of trials. This last result is a rebuttal exhibit, not main statistics: it is engagement evidence (the model engaged the malicious content), not a confidentiality result, and `c_sealed`’s 0% is partly true-by-construction.

The honest envelope is narrower than the headline. External validity rests on *two* of three pretraining lineages (Qwen and Mistral); the registered ≥ 3 -lineage bar is not closed. The RQ1 leak and the formal H2 contrast are carried by the single capable Mistral anchor, not by the 1.7B (which is sub-floor by design at 18.33% Control bypass and is the weak-end apparatus, not the headline victim). The registered powered 2pp non-inferiority TOST clustered by `task_id` was run on neither victim, so non-inferiority (H4) is not formally established anywhere – at most a point-estimate ordering is licensed (Mistral blind-c legit-success 78.85% [72.66, 84.19] above Control 73.56%); “0 invalid outputs” is likewise not literally met on vLLM (regular-c 98.9% valid, `c_closed_d_r3` 99.9%, `c_guided_d_r3` 97.8%, all from free-string truncation under Kleene $*$ that fails safe). The C-guided superiority result (H6) – bypass 0, deflection 0, and legit-success 100.0% (624/624) above Control’s 58.3% – holds only on the 1.7B, and grammar-compile time (H5) is uncaptured in all three DBs. The single subject system (Tantalus) makes this a feasibility claim, not a population-level one; the corpus is selected-on-outcome (the live-malware analogue), so bypass rates are conditional on attack potency and large N tightens confidence intervals [8] without buying generalization. The class-level “eliminates” claim is carried by the soundness theorem, which quantifies over all logit distributions and so holds against a fully injection-compromised model, not by any sample; proof and evidence stay in separate lanes.

Future work. The open items follow directly from this envelope:

- **A third pretraining lineage** to close the ≥ 3 -lineage bar (§8). Nemotron-3-Super-120B is cached and ready; a full 11-condition ladder on a Gemma/Llama/Nemotron victim would replicate H1/H2/H3/H6 cross-family.
- **Capable-anchor C-guided and C+ cells.** The Mistral run used blind `c` only; running `c_guided` and `c_plus` on a capable model would extend the H6 superiority result and the closed-response validity result beyond the weak 1.7B.
- **The powered 2pp non-inferiority TOST** clustered by `task_id` at `--legit-rounds 7` ($\approx 1,456$ trials/cond), to formally establish H4 rather than the current point-estimate ordering.
- **An H5 grammar-compile-time probe** (xgrammar [6] / guidance FSM compile latency, p99 against the 25 ms envelope), an out-of-band measurement requiring a live engine.
- **The full mechanized soundness proof** as a companion artifact, promoting the proof sketch’s caveats to discharged hypotheses and making the for-all-logit lemma and the tokenizer-lifting argument first-class.
- **Paper-2: free-text confidentiality** – enumerable-response `c_plus` where the response space is enumerable, plus layered DLP for genuinely open-vocabulary channels (`searchFiles.query`, composed `respondToUser.message`) that lie outside this paper’s action-integrity scope.

The thesis, restated, is structural and narrow: enforcement at the decoding boundary – the grammar mask applied at the sampler before any token is drawn – is positive security all the way down to the generative act. The transformer’s logits encode intent and may be arbitrarily compromised; the generated tokens are constrained regardless, so the unauthorized action is not caught but ungenerable, with emitted $\in L(G_s) \subseteq \text{Authorized}(s)$.

References

- [1] M. Galloway. *Application Whitelisting as a Malicious Code Protection Control*. 2020.
- [2] J. N. Amaral et al. *About Computing Science Research Methodology*. Methodology primer.
- [3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [4] H. Zhang et al. Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. arXiv preprint, 2024.
- [5] B. T. Willard and R. Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.
- [6] Y. Dong et al. XGrammar: Flexible and efficient structured generation engine for large language models. arXiv:2411.15100, 2024.
- [7] A. Ardit, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [8] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.
- [9] G. Gerganov and contributors. llama.cpp: LLM inference in C/C++ with GBNF grammar-constrained sampling. Software. <https://github.com/ggml-org/llama.cpp>.